CUESTIONARIO 7

Herencia y polimorfismo

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué imprime este programa de herencia?

```
class Padre {
    public Padre() {
        System.out.println("Clase Padre");
    }
}
class Hija extends Padre {
    public Hija() {
        System.out.println("Clase Hija");
    }
}
public class Test {
    public static void main(String[] args) {
        new Hija();
    }
}
```

Seleccione una:

- a. Error de compilación
- b. Clase Hija
- c. Clase Hija Clase Padre
- d. Clase Padre Clase Hija ✓

Justificación: El constructor de Hija llama implícitamente a super() como primera instrucción, así que primero se ejecuta el constructor de Padre y después el cuerpo del de Hija. La opción c invierte el orden en que se encadenan los constructores.

Fuente: Quiz APX #8

La respuesta correcta es: Clase Padre Clase Hija

Mi justificación
Para construir Hija necesitas construir padre primero así que se ejecutan los constructores Padre e Hija respectivamente.

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Given the following classes, which line of code correctly creates an instance of the Dog class?

```
class Animal {
    // ...
}
class Dog extends Animal {
    // ...
}
```

Seleccione una o más de una:

- [x] a. Dog dog = new Dog(); ✓
- [x] b. Animal dog = new Dog(); ✓
- [ ] c. Dog dog = new Animal();
- [ ] d. Animal dog = new Animal();

Justificación: a y b crean un objeto Dog (new Dog()); b lo guarda en una referencia de la superclase, lo cual es válido por polimorfismo. c no compila, porque un Animal no es un Dog, y d compila pero crea un Animal, no un Dog.

Fuente: Java.pdf #123 · también aparece en: Cuestionario APX V3 #22

Las respuestas correctas son: Dog dog = new Dog();, Animal dog = new Dog();

Mi justificación
La clase Dog es hija de Animal, así que podemos crear una referencia Animal y Dog que apunte a un objeto new Dog. Si usamos new Animal() estamos creando un Animal, no un Dog.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which statement, when inserted into line '' // TODO code application logic here", is valid in compilation time change?

```
public class SampleClass {
    public static void main(String[] args) {
        AnotherSampleClass asc = new AnotherSampleClass();
        SampleClass sc = new SampleClass();
        // TODO code application logic here
    }
}
class AnotherSampleClass extends SampleClass { }
```

Seleccione una:

- a. asc = sc;
- b. sc = asc; ✓
- c. asc = (Object) sc;
- d. asc= sc.clone();

Justificación: Asignar un objeto de la subclase a una referencia de la superclase (upcasting) es implícito y siempre compila: sc = asc. Las demás opciones intentan asignar un tipo más general (SampleClass u Object, que es lo que devuelve clone()) a una referencia de la subclase sin cast.

Fuente: Java Sesiones #39

La respuesta correcta es: sc = asc;

Mi justificación
Un padre solo puede apuntar a un hijo y no viceversa, entonces las opciones que contengan asc = sc están mal. La única válida es sc = asc.

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál será el resultado de ejecutar el siguiente código?

```
class Parent {
    public void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    public void display() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.display();
    }
}
```

Seleccione una:

- a. Parent
- b. Error de ejecución
- c. Child ✓
- d. Error de compilación

Justificación: Aunque la referencia sea de tipo Parent, el objeto es un Child, y los métodos de instancia se resuelven en tiempo de ejecución (despacho dinámico). Por eso se ejecuta el display() sobrescrito de Child. 'Parent' solo sería correcto si display() fuera static (ocultamiento en lugar de sobrescritura).

Fuente: Java.pdf #21 · también aparece en: Simulación APX V4 #9

La respuesta correcta es: Child

Mi justificación
Crear un new Child() hace que el método dominante sea el del hijo, así que obj.display() imprime Child, si hubiéramos querido Parent ocupamos super.display().

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál será el resultado del siguiente fragmento de código?

```
class Animal {
    public void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound();
    }
}
```

Seleccione una:

- a. Animal sound
- b. Error de ejecución
- c. Error de compilación
- d. Bark ✓

Justificación: La llamada a un método de instancia se resuelve en tiempo de ejecución según el tipo real del objeto (Dog), que sobrescribe makeSound(), así que se imprime 'Bark'. 'Animal sound' sería el resultado si makeSound fuera static o si Dog no lo sobrescribiera.

Fuente: Java.pdf #211 · también aparece en: Cuestionario APX V3 #6

La respuesta correcta es: Bark

Mi justificación
Un objeto new Dog hace override al método makeSound así que imprime Bark.

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál será la salida del siguiente código?

```
class Animal {
    public void sonido() {
        System.out.println("Sonido genérico de animal.");
    }
}

class Perro extends Animal {
    @Override
    public void sonido() {
        System.out.println("Ladrido.");
    }
}

public class TestHerencia {
    public static void main(String[] args) {
        Animal miAnimal = new Perro();
        miAnimal.sonido();
    }
}
```

Seleccione una:

- a. Sonido genérico de animal.
- b. Ladrido. ✓
- c. Error de compilación.
- d. No se imprime nada.

Justificación: La llamada se resuelve en tiempo de ejecución según el tipo real del objeto (Perro), que sobrescribe sonido(), por eso se imprime 'Ladrido.'. La opción a solo sería cierta si el método no estuviera sobrescrito o fuera static.

Fuente: Quiz Desarrollo APX 2 #11

La respuesta correcta es: Ladrido.

Mi justificación
Lo mismo que la pregunta 5. Crer un new Perro() hace que sonido() imprima Ladrido porque hace override del método.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Que muestra en pantalla al ejecutar este código?

```
class Parent {
    public void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    public void display() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        ((Child) obj).display();
    }
}
```

Seleccione una:

- a. Child ✓
- b. Parent
- c. Compilation error
- d. Runtime error

Justificación: obj apunta a un Child, así que el cast (Child) es válido en tiempo de ejecución, y display() está sobrescrito, de modo que se ejecuta la versión de Child. El cast no cambia qué método se invoca (despacho dinámico), por eso no se imprime Parent.

Fuente: Simulación APX V4 #2 · también aparece en: Java.pdf #10

La respuesta correcta es: Child

Mi justificación
Lo mismo que la pregunta 6. Incluso haciendo casteo no altera el override de display.

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es el resultado del siguiente código? (SampleClass y AnotherSampleClass están en el paquete por defecto y AnotherSampleClass extends SampleClass.)

```
SampleClass sc = new SampleClass();
AnotherSampleClass asc = new AnotherSampleClass();
System.out.println("sc: " + sc.getClass());
System.out.println("asc: " + asc.getClass());
```

Seleccione una:

a. sc: class Object asc: class AnotherSampleClass

b. sc: class SampleClass asc: class AnotherSampleClass ✓

c. sc: class AnotherSampleClass asc: class SampleClass

d. sc: class AnotherSampleClass asc: class AnotherSampleClass

Justificación: getClass() devuelve la clase real del objeto en tiempo de ejecución, y al imprimirla aparece 'class ' + nombre. sc apunta a un SampleClass y asc a un AnotherSampleClass, así que cada uno se imprime con su propia clase. La opción a confunde con Object: getClass() solo devuelve Object si el objeto es literalmente un new Object(). Compárala con la variante que añade 'sc = asc;': ahí sc apuntaría al objeto AnotherSampleClass y se imprimiría 'sc: class AnotherSampleClass'.

Nota sobre el material original: El material original muestra solo el fragmento, sin las declaraciones de las clases. Se añadió al enunciado que ambas están en el paquete por defecto y que AnotherSampleClass hereda de SampleClass. Si estuvieran dentro de un paquete, getClass() imprimiría el nombre calificado (class paquete.SampleClass).

Fuente: Quiz Desarrollo APX 1 #5

La respuesta correcta es: sc: class SampleClass asc: class AnotherSampleClass

Mi justificación
No hay confusión, se imprime la clase a la que pertenece acada uno, SampleClass y AnotherSampleClass respectivamente.

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
1  public class SampleClass {
2      public static void main(String[] args) {
3          AnotherSampleClass asc = new AnotherSampleClass();
4          SampleClass sc = new SampleClass();
5          sc = asc;
6          System.out.println("sc: " + sc.getClass());
7          System.out.println("asc: " + asc.getClass());
8      }
9  }
10
11 class AnotherSampleClass extends SampleClass {}
```

Seleccione una:

- a. sc: class SampleClass asc: class AnotherSampleClass
- b. sc: class Object asc: class AnotherSampleClass
- c. sc: class AnotherSampleClass asc: class SampleClass
- d. sc: class AnotherSampleClass asc: class AnotherSampleClass ✓

Justificación: getClass() devuelve la clase real del objeto en tiempo de ejecución, no el tipo declarado de la referencia. Tras sc = asc, las dos referencias apuntan al mismo objeto AnotherSampleClass. La opción a es el distractor típico que confunde el tipo de la variable con el tipo del objeto.

Fuente: Java E #8 · también aparece en: Java DE #17, Java D #16, Java Sesiones #48

La respuesta correcta es: sc: class AnotherSampleClass asc: class AnotherSampleClass

Mi justificación
Al hacer sc = asc, sc apunta ahora a un objeto de clase AnotherSampleClass así que ambos imprimen AnotherSampleClass.

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es el resultado del siguiente código?

```
public class SampleClass {
    public static void main(String[] args) {
        SampleClass sc, scA, scB;
        sc = new SampleClass();
        scA = new SampleClassA();
        scB = new SampleClassB();
        System.out.println("Hash is: " + sc.getHash() + ", " +
                scA.getHash() + ", " + scB.getHash());
    }

    public int getHash() {
        return 111111;
    }
}

class SampleClassA extends SampleClass {
    @Override
    public int getHash() {
        return 44444444;
    }
}

class SampleClassB extends SampleClass {
    @Override
    public int getHash() {
        return 999999999;
    }
}
```

Seleccione una:

a. La compilación falla.

b. Se lanza una excepción en tiempo de ejecución.

c. Hash is: 111111, 44444444, 999999999 ✓

d. No hay resultado porque este no es el método correcto para determinar el hash.

Justificación: SampleClassA y SampleClassB sobrescriben getHash(). Por polimorfismo, aunque scA y scB se declaran como SampleClass, se ejecuta la versión de la clase real del objeto. El código compila y no lanza excepciones. getHash() es un método propio y válido; no tiene nada que ver con hashCode(), así que la opción d es falsa.

Fuente: Quiz Desarrollo APX 1 #6

La respuesta correcta es: Hash is: 111111, 44444444, 999999999

Mi justificación
No hay confusión, cada HashCode() se va actualizando en cada extensión de clase SampleClass y devuelve el valor que tenga su método con override. Entonces tal cual imprime 111111, 4444444, 999999.

Pregunta 11

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What will be the output of the following code snippet?

```
class A {
    public A() {
        System.out.println("A's constructor");
    }
}
class B extends A {
    public B() {
        super();
        System.out.println("B's constructor");
    }
}
class C extends B {
    public C() {
        System.out.println("C's constructor");
    }
}
public class Test {
    public static void main(String[] args) {
        C c = new C();
    }
}
```

Seleccione una:

- a. A's constructor B's constructor C's constructor ✓
- b. C's constructor A's constructor B's constructor
- c. A's constructor C's constructor B's constructor

Justificación: Todo constructor llama primero a super(): en C es implícito y en B es explícito. Por eso los cuerpos se ejecutan desde la superclase hacia abajo: A, luego B y luego C. Los distractores suponen que el cuerpo de C se ejecuta antes que la cadena de super().

Fuente: Java.pdf #88

La respuesta correcta es: A's constructor B's constructor C's constructor

Mi justificación
Al ser B una extensión de A y C una extensión de B, crear una instancia de C ejecuta en orden los constructores de A, luego B y luego C. Entonces imprime A constructor, B constructor, C constructor en ese orden.

Pregunta 12

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Que imprime en pantalla este código?

```
class Base {
    public Base() {
        System.out.println("Base constructor");
    }
    public Base(String message) {
        System.out.println("Base constructor with message: " + message);
    }
}
class Derived extends Base {
    public Derived() {
        super("Hello");
        System.out.println("Derived constructor");
    }
}
public class Test {
    public static void main(String[] args) {
        Derived derived = new Derived();
    }
}
```

Seleccione una:

a. It will print "Base constructor" followed by "Derived constructor"

b. It will print "Base constructor with message: Hello" followed by "Derived constructor" ✓

c. It will throw a compilation error.

d. It will print "Derived constructor" followed by "Base constructor with message: Hello"

Justificación: super("Hello") elige el constructor de Base que recibe un String, y este se ejecuta antes del resto del constructor de Derived. El constructor sin argumentos de Base no se llama porque ya hay una llamada explícita a super(...).

Fuente: Simulación APX V4 #14 · también aparece en: Java.pdf #32

La respuesta correcta es: It will print "Base constructor with message: Hello" followed by "Derived constructor"

Mi justificación
El constructor de Derived() invoca al constructor Base(String) de Base con super("Hello") así que primero se imprime Base constructor with message Hello y luego el print de Derived constructor.

Pregunta 13

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
class Alpha { String getType() { return "alpha"; } }
class Beta extends Alpha { String getType() { return "beta"; } }
public class Gamma extends Beta { String getType() { return "gamma"; }
    public static void main(String[] args) {
        Gamma g1 = new Alpha();
        Gamma g2 = new Beta();
        System.out.println(g1.getType() + " " + g2.getType());
    }
}
```

Seleccione una:

- a. Alpha beta ⊗
- b. Beta beta.
- c. Gamma gamma.
- d. Compilation fails.

Justificación: No se puede asignar un objeto de una superclase (Alpha o Beta) a una referencia de la subclase Gamma sin un cast explícito, así que el código no compila. Los distractores suponen un polimorfismo en sentido inverso, que Java no permite.

Fuente: Java Sesiones #10

La respuesta correcta es: Compilation fails.

Mi justificación
Crear un Alpha ejecuta el método getType() de Alpha, y crear un Beta ejecuta el método getType() de Beta, entonces en orden imprime Alpha Beta.

Pregunta 14

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What will happen when you try to compile the following code?

```
class Parent {
    protected void show() {
        System.out.println("Parent show");
    }
}
class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child show");
    }
}
public class Test {
    public static void main(String[] args) {
        Child child = new Child();
        child.show();
    }
}
```

Seleccione una:

- a. It will print "Parent show".
- b. It will print "Child show". ⊗
- c. Compilation error.
- d. It will print "Parent show" followed by "Child show".

Justificación: Al sobrescribir un método no se puede reducir su visibilidad. Parent.show() es protected y Child.show() tiene acceso de paquete, que es más restrictivo, así que el código no compila. Si Child.show() fuera protected o public, imprimiría "Child show".

Fuente: Java.pdf #166

La respuesta correcta es: Compilation error.

?

Mi justificación
Este es directo, si creamos un Child, imprime el método show del child, entonces imprime Child show.

Pregunta 15

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
class CA {
    CA() {
        System.out.print("middle");
    }
    CA(String s) {
        System.out.print(s);
    }
}
class CB extends CA {
}
public class Test {
    public static void main(String[] args) {
        new CA("message ");
        new CB();
        System.out.print(" end");
    }
}
```

Seleccione una:

- a. message middle end ✓
- b. An exception is thrown at runtime.
- c. Compilation fails
- d. message end middle

Justificación: new CA("message ") imprime "message ". CB no declara constructor, así que recibe uno por defecto que llama a super(), es decir, a CA(), que imprime "middle". Al final se imprime " end". El código compila porque CA sí tiene un constructor sin argumentos.

Fuente: Java.pdf #103 · también aparece en: Simulación APX V4 #35

La respuesta correcta es: message middle end

Mi justificación
En orden se ejecutan: constructor CA(String) -> (Constructor CB que aparentemente no existe pero ejecuta primero Constructor CA()) -> Se imprime end. Siguiendo esa secuencia obtenemos “message middle end”.

Pregunta 16

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which three methods, inserted individually at line, will correctly complete class Two(Choose three)?

```
10.    class One {
11.        void foo() { }
12.    }
13.    class Two extends One {
14.        //insert method here
15.    }
```

Seleccione una o más de una:

- [ ] a. int foo() { /* more code here */ }
- [x] b. void foo() { /* more code here */ } ✓
- [x] c. public void foo() { /* more code here */ } ✓
- [ ] d. private void foo() { /* more code here */ }
- [x] e. protected void foo() { /* more code here */ } ✓

Justificación: Al sobrescribir se puede mantener el acceso de paquete (sin modificador) o ampliarlo a protected o public, pero no reducirlo a private. Tampoco se puede cambiar el tipo de retorno de void a int con la misma firma: no es sobrecarga (los parámetros son iguales) ni una sobrescritura válida.

Fuente: Java Sesiones #33

Las respuestas correctas son: void foo() { /* more code here */ }, public void foo() { /* more code here */ }, protected void foo() { /* more code here */ }

Mi justificación
Los métodos a los que heredemos deben tener un modificador de acceso igual o más accesible que el método en el padre, así que agregarle private limitaría su alcance, por lo que es inválido. No podemos cambiar el valor de retorno de foo() así que int foo() es inválido. Las demás cumplen con el primer enunciado así que b c y e son las correctas.

Pregunta 17

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Qué métodos de anulación (Override) son válidos para las siguientes clases?

```
class Parent {
    void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    // Override here
}
```

Seleccione una o más de una:

- [ ] a. static void display() { System.out.println("Child"); }
- [ ] b. private void display() { System.out.println("Child"); }
- [x] c. void display() { System.out.println("InChild"); }
- [x] d. public void display() { System.out.println("Child"); }

Justificación: Sobrescribir exige la misma firma y no reducir la visibilidad. c mantiene el acceso de paquete y d lo amplía a public, así que ambas son válidas. a no compila porque un método static no puede ocultar a un método de instancia. b no compila porque private reduce la visibilidad.

Fuente: Java.pdf #127 · también aparece en: Cuestionario APX V3 #24

Las respuestas correctas son: void display() { System.out.println("InChild"); }, public void display() { System.out.println("Child"); }

Mi justificación
Los métodos a los que heredemos deben tener un modificador de acceso igual o más accesible que el método en el padre, entonces void display cuenta con si mismo y con public void display().

Pregunta 18

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which line of code, when inserted at the commented line, causes the compiler to fail?

```
class ABC {
    public String met() {
        return "snippet 1";
    }
}
public class ABCD extends ABC {
    // Insert code here
}
```

Seleccione una:

a. public String met() { return "4"; }
b. public char met(int i) { return '3';}
c. public String met(int i) { return "5"; }
d. public char met() { return '2'; } ✓

Justificación: Con la misma firma (met sin parámetros) se trata de una sobrescritura, y el tipo de retorno debe ser igual o covariante. char no es subtipo de String, así que d no compila. b y c cambian la lista de parámetros, por lo que son sobrecargas válidas aunque cambien el tipo de retorno.

Fuente: Java.pdf #102

La respuesta correcta es: public char met() { return '2'; }

Mi justificación
Si haces override de un método debes respetar el tipo de valor de retorno, entonces String met() no puede heredarse como char met(). Nota: met(int i) es otro método diferente, por lo que está permitido poner char como su valor de retorno.

Pregunta 19

Parcialmente correcta

Se puntúa 0,80 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál de las siguientes afirmaciones describe con precisión las relaciones que pueden existir entre clases en Java?

Seleccione una o más de una:

- [x] a. La herencia representa una relación "es-un" donde una clase deriva de otra clase.
- [x] b. La composición representa una relación "tiene-un" donde una clase contiene una instancia de otra clase.
- [ ] c. Se debe preferir la herencia sobre la composición para fomentar la reutilización de código y la flexibilidad.
- [x] d. Tanto la herencia como la composición pueden usarse juntas para modelar relaciones complejas.
- [ ] e. El uso (o asociación) representa una relación "usa-un" donde una clase utiliza métodos o instancias de otra clase.
- [x] f. Se debe preferir la composición sobre la herencia para fomentar la reutilización de código y la flexibilidad.

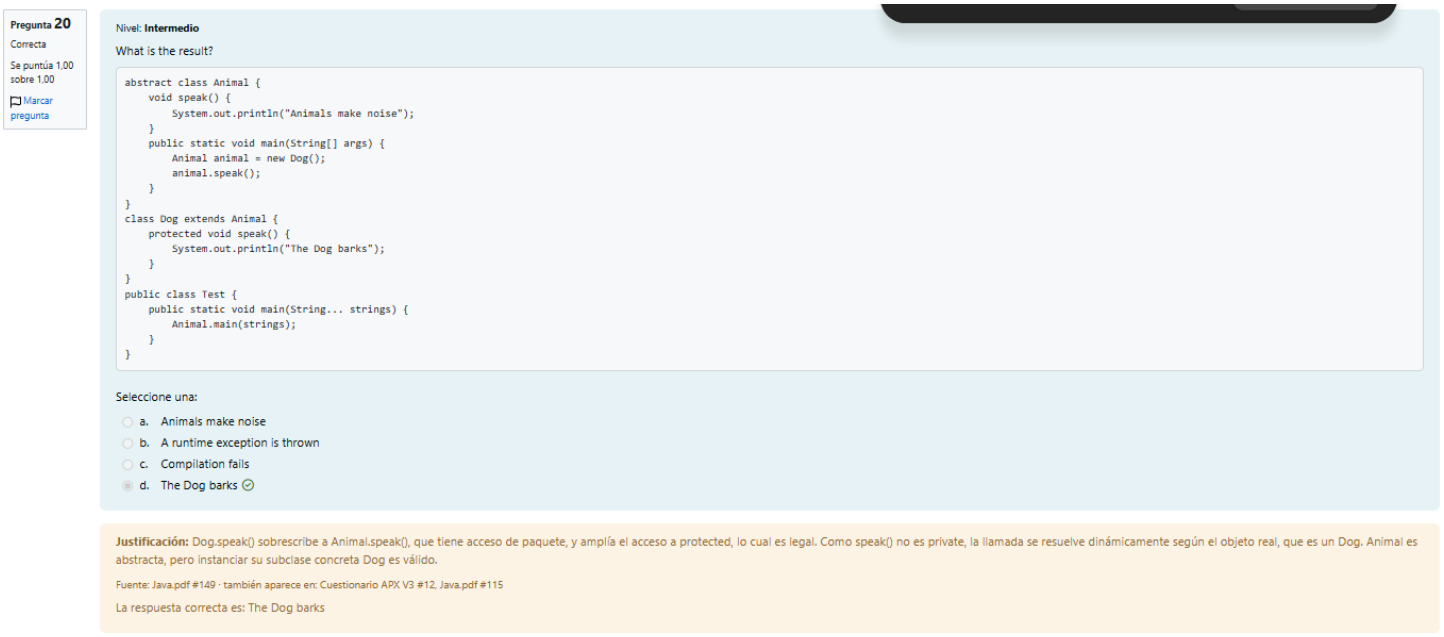
Ha seleccionado correctamente 4.

Justificación: La herencia modela «es-un», la composición «tiene-un» y la asociación o uso «usa-un», y la herencia y la composición pueden combinarse. La recomendación clásica es preferir la composición sobre la herencia (f), porque reduce el acoplamiento. Por eso c, que dice lo contrario, es la única falsa.

Fuente: Simulación APX V2 #13 · también aparece en: Java.pdf #83

Las respuestas correctas son: La herencia representa una relación "es-un" donde una clase deriva de otra clase., La composición representa una relación "tiene-un" donde una clase contiene una instancia de otra clase., Tanto la herencia como la composición pueden usarse juntas para modelar relaciones complejas., El uso (o asociación) representa una relación "usa-un" donde una clase utiliza métodos o instancias de otra clase., Se debe preferir la composición sobre la herencia para fomentar la reutilización de código y la flexibilidad.

Mi justificación
Las únicas que no son ciertas es la c) ya que la composición sobre la herencia hace que sea más fácil implementar clases y reutilizarlas en otros proyectos; y la e) ya que no existe “usa-un” solo “es-un” y “tiene-un”.



Pregunta 20

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
abstract class Animal {
    void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Seleccione una:

a. Animals make noise

b. A runtime exception is thrown

c. Compilation fails

d. The Dog barks

Justificación: Dog.speak() sobrescribe a Animal.speak(), que tiene acceso de paquete, y amplía el acceso a protected, lo cual es legal. Como speak() no es private, la llamada se resuelve dinámicamente según el objeto real, que es un Dog. Animal es abstracta, pero instanciar su subclase concreta Dog es válido.

Fuente: Java.pdf #149 · también aparece en: Cuestionario APX V3 #12, Java.pdf #115

La respuesta correcta es: The Dog barks

Mi justificación
El método main de Animal es static así que se puede llamar sin crear una instancia de Animal, este crea un Perro con el método speak() con override válido (protected es más accesible que default). Entonces ejecuta el método speak() de Dog, el cual imprime “The dog barks”.

Pregunta 21

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
abstract class Animal {
    public void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}

class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}

public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Seleccione una:

- a. Compilation fails. ✓
- b. The Dog barks
- c. A runtime exception is thrown.
- d. Animals make noise

Justificación: Al sobrescribir un método no se puede reducir su visibilidad. speak() es public en Animal y Dog lo declara protected, así que el código no compila. Si Dog.speak() fuera public, se imprimiría "The Dog barks".

Fuente: Java.pdf #189 · también aparece en: Cuestionario APX V3 #20

La respuesta correcta es: Compilation fails.

Mi justificación
La compilación falla porque el método speak() en el padre es public y en el hijo es protected, ósea que el modificador de acceso es menos accesible, lo cual causa error de compilación.

Pregunta 22
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código. ¿Cuál será el resultado?

```
import java.util.*;

class Alpha{
    public String mensaje (String msj) {
        return msj;
    }
}
class Beta extends Alpha {
    public String mensaje (String msj) {
        return msj.replace('a', 'e');
    }
}
class Gamma extends Beta{
    public String mensaje (String msj) {
        return msj.substring(2);
    }
}
public class MultiHerencia {
    public static void main(String[] args) {
        List<Alpha>list = new ArrayList<Alpha>();
        list.add(new Alpha());
        list.add(new Beta());
        list.add(new Gamma());
        for (Alpha a: list) {
            System.out.println(a.mensaje("Java"));
        }
    }
}
```

Seleccione una:

a. Java Jeve ve
b. Java Java Java
c. Java Jeve va ✓
d. Compilation fails

Justificación: Cada objeto usa su propia versión sobrescrita de mensaje: Alpha devuelve «Java», Beta reemplaza las a por e («Jeve») y Gamma devuelve substring(2) de «Java», es decir, «va». Gamma no aplica el replace de Beta porque no llama a super. El distractor a («ve») supone que Gamma encadena la lógica de Beta. Cada palabra se imprime en su propia línea.

Nota sobre el material original: En el material original el código no incluía el import de List y ArrayList (así no compila). Se añadió «import java.util.*;» porque la intención de la pregunta es evaluar la sobrescritura, no los imports.

Fuente: Quiz Desarrollo APX 2 #18

La respuesta correcta es: Java Jeve va

Mi justificación
En orden de la lista list: Alpha.mensaje(“Java”) imprime tal cual “Java” Beta.mensaje(“Java”) remplaza cada a con e así que imprime “Jeve” Gamma.mensaje(“Java”) imprime desde la tercer letra (índice 2) en adelante, así que imprime solo “va”

Pregunta 23

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Consider the following Java Code: What will be the output of the above code?

```
class Animal {
    public void makeSound() {
        System.out.println("Sound");
    }
    public void makeSound(String sound) {
        System.out.println(sound);
    }
}
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
    public void makeSound(int times) {
        for (int i = 0; i < times; i++) {
            System.out.println("Bark");
        }
    }
}
public class Test {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Animal dog = new Dog();
        animal.makeSound();
        animal.makeSound("Roar");
        dog.makeSound();
        dog.makeSound("Growl");
    }
}
```

Seleccione una:

a. Sound Roar Bark Roard

b. Sound Roar Bark Growl ✓

c. Sound Roar Bark Bark

d. Sound Roar Sound Growl

Justificación: dog tiene tipo estático Animal, pero el objeto es un Dog. makeSound() está sobrescrito y se resuelve dinámicamente a Dog, que imprime "Bark". makeSound("Growl") solo existe en Animal (la sobrecarga de Dog es makeSound(int)), así que imprime "Growl". La opción d ignora la resolución dinámica de la versión sin parámetros.

Fuente: Java.pdf #73

La respuesta correcta es: Sound Roar Bark Growl

Mi justificación
Este es bastante directo: Ejecutar animal.makeSound() ejecuta el método sin parámetro de Animal, así que imprime “Sound”. Ejecutar animal.makeSound(“Roar”) ejecuta el método con parámetro de Animal, así que imprime el parámetro “Roar”. Ejecutar dog.makeSound() ejecuta el método sin parámetro de Dog, así que imprime “Bark”. Ejecutar dog.makeSound(“Growl”) ejecuta el método con parámetro de Animal ya que no fue sobreescrito en la definición de Dog, así que imprime el parámetro “Growl”.

Pregunta 24

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
class Feline {
    public String type = "f";

    public Feline() {
        System.out.print("feline ");
    }
}

public class Cougar extends Feline {
    public Cougar() {
        System.out.print("cougar ");
    }

    void go() {
        type = "c";
        System.out.print(this.type + super.type);
    }

    public static void main(String[] args) {
        new Cougar().go();
    }
}
```

Seleccione una:

a. Cougar c f

b. Compilation fails

c. Feline cougar c c ✓

d. Feline cougar c f

Justificación: El constructor de Cougar llama implícitamente a super(), así que primero se imprime "feline " y luego "cougar ". Cougar no redeclara type, de modo que this.type y super.type son el mismo campo heredado. Al asignarle "c", ambos imprimen c. La opción d (c f) es el distractor de quien cree que super.type es un campo distinto.

Fuente: Java E #12 · también aparece en: Java Sesiones #7

La respuesta correcta es: Feline cougar c c

Mi justificación
Instanciar un Cougar ejecuta primero el constructor de su padre Feline y luego el constructor de Cougar, entonces primero imprime “feline cougar”. Luego, al ejecutar el método go() de cougar, se define type = “c” y es importante ver que no se incializa un type nuevo para cougar, porque eso sería con “char type = “c”” sino que se altera el de su padre (para esa instancia). Por lo tanto, tanto this.type como super.type valen c, así que imprime “feline cougar c c”.

Pregunta 25

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál de las siguientes anotaciones es verdadera acerca de la siguiente jerarquía de clases?

```
class Vehicle {
    String type = "Vehicle";
}
class Car extends Vehicle {
    String type = "Car";
}
class ElectricCar extends Car {
    String type = "ElectricCar";
}
```

Seleccione una:

- a. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar ✓
- b. Car myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle
- c. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle
- d. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Car

Justificación: Los atributos no son polimórficos: se ocultan (hiding), y el campo al que se accede depende del tipo de la referencia. Con una referencia ElectricCar se lee "ElectricCar". La opción b es tentadora, pero con una referencia Car se lee el campo de Car ("Car"), no "Vehicle".

Fuente: Java.pdf #157

La respuesta correcta es: ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar

Mi justificación
Si ponemos myCar.type, obtenemos la clase del objeto con el que fue construido el objeto. Ya que los 3 tienen su propio type, ninguno tendría el type del padre, entonces el único válido es ElectricCar con sálida ElectricCar.

Pregunta 26

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
11. class Person {
12. String name = "No name";
13. public Person (String nm) { name = nm; }
14. }
15.
16. class Employee extends Person {
17.  String empID = "0000";
18.  public Employee (String id) { empID = id; }
19. }
20.
21. public class EmployeeTest {
22.  public static void main(String[] args) {
23.   Employee e = new Employee("4321");
24.   System.out.println(e.empID);
25.  }
26. }
```

Seleccione una:

- a. 4321. ⊗
- b. 0000.
- c. An exception is thrown at runtime.
- d. Compilation fails because of an error in line 18.

Justificación: El constructor de Employee no llama explícitamente a super(...), así que el compilador inserta super(). Pero Person solo tiene un constructor que recibe un String, por eso la compilación falla en la línea 18. Java no genera un constructor sin argumentos cuando la clase ya declara otro.

Fuente: Java Sesiones #51 · también aparece en: Java E #6

La respuesta correcta es: Compilation fails because of an error in line 18.

Mi justificación
Este es bastante directo, se crea un empleado con el constructor Employee(String id) y e.empID vale 4321, así que se imprime 4321.

Pregunta 27

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
class Atom {
    Atom() { System.out.print("atom "); }
}
class Rock extends Atom {
    Rock(String type) { System.out.print(type); }
}
public class Mountain extends Rock {
    Mountain() {
        super("granite ");
        new Rock("granite ");
    }
    public static void main(String[] a) { new Mountain(); }
}
```

Seleccione una:

- a. Compilation fails.
- b. Atom granite.
- c. Granite granite.
- d. Atom granite granite.
- e. An exception is thrown at runtime.
- f. Atom granite atom granite. ✓

Justificación: super("granite ") ejecuta primero el constructor implícito de Atom ("atom ") y luego imprime "granite ". Después, new Rock("granite ") vuelve a encadenar Atom() y se imprime otra vez "atom granite ". El distractor d olvida que cada nuevo Rock invoca de nuevo el constructor de Atom.

Fuente: Java Sesiones #4

La respuesta correcta es: Atom granite atom granite.

Mi justificación
Crear una instancia de Mountain primero crearía una de atom y luego de Rock, pero el constructor de Mountain() aclara con super(“granite”) que el constructor que se usará será el de Rock(String type), por lo tanto no necesitamos el constructor sin parámetros Rock(). Entonces, en orden: Super(“granite”) = Atom -> granite New Rock(“granite”) = Atom -> granite (lo mismo que el anterior) Al final tenemos atom granite atom granite.

Pregunta 28
Correcta
Se puntúa 1.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
class X {
    X() {
        System.out.print(1);
    }
    X(int x) {
        this();
        System.out.print(2);
    }
}

public class Y extends X {
    Y() {
        super(6);
        System.out.print(3);
    }
    Y(int y) {
        this();
        System.out.print(4);
    }

    public static void main(String[] a) {
        new Y(5);
    }
}
```

Seleccione una:

- a. 1234 ✓
- b. 2143
- c. 4321
- d. 134
- e. 13
- f. 2134

Justificación: new Y(5) llama a this(), es decir, a Y(). Y() llama a super(6), es decir, a X(int), que a su vez llama a this(), es decir, a X(). Cada constructor termina de delegar antes de imprimir lo suyo, así que al deshacerse la pila de llamadas se imprimen 1, 2, 3 y 4: la salida es 1234 (verificada con JDK 21). NOTA DE FUENTE: ninguna de las opciones del material original (1213, 2143, 4321, 134, 13, 2134) era la salida real.

Nota sobre el material original: En el material original ninguna opción era la salida real del programa (1234). Se sustituyó la opción a (1213) por 1234.

Fuente: Java E #18

La respuesta correcta es: 1234

Mi justificación
Al construir Y(5) ejecutamos Y(int Y), el cual ejecuta this() ósea Y() el cual ejecuta super(6) ósea X(int x) el cual ejecuta this() ósea X() el cual imprime 1. Siguiendo esta secuencia en reversa, tenemos que primero imprime 1, luego imprime 2, luego continua con lo restante de Y() que es imprimir 3 y finalmente continua con Y(int Y) que es imprimir 4. Al final tenemos 1234.

Pregunta 29

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

What should statement1, statement2, and statement3, be respectively, in order to produce the result?

Shape: constructor
Shape: foo
Square: foo

```
public class SuperTest {
    public static void main(String[] args) {
        //statement1
        //statement2
        //statement3
    }
}
class Shape {
    public Shape() {
        System.out.println("Shape: constructor");
    }
    public void foo() {
        System.out.println("Shape: foo");
    }
}
class Square extends Shape {
    public Square() {
        super();
    }
    public Square(String label) {
        System.out.println("Square: constructor");
    }
    public void foo() {
        super.foo();
    }
    public void foo(String label) {
        System.out.println("Square: foo");
    }
}
```

Seleccione una:

a. Square square = new Square ("bar"); square.foo ("bar"); square.foo();
b. Square square = new Square ("bar"); square.foo ("bar"); square.foo ("bar");
c. Square square = new Square (); square.foo (); square.foo(bar);
d. Square square = new Square (); square.foo (); square.foo("bar"); ✓
e. Square square = new Square (); square.foo (); square.foo ();

Justificación: new Square() llama a super() e imprime "Shape: constructor". Después, square.foo() delega en super.foo() e imprime "Shape: foo", y square.foo("bar") imprime "Square: foo". Las opciones con new Square("bar") imprimen además "Square: constructor" (el super() implícito imprime antes "Shape: constructor"). La opción e imprime "Shape: foo" dos veces, y la c no compila porque bar no está declarada.

Nota sobre el material original: En el formulario original el enunciado decía solo «What is the result?», sin la salida esperada; se completó con el enunciado y la salida que muestra la imagen de la pregunta.

Fuente: Java Sesiones #23 · también aparece en: Quiz Desarrollo APX 2 #17

La respuesta correcta es: Square square = new Square (); square.foo (); square.foo("bar");

Mi justificación
Shape: constructor. Necesitamos ejecutar el constructor Shape() Shape: foo Necesitamos ejecutar foo() de la clase Shape Square: foo Necesitamos ejecutar foo() de la clase Square. Desde square, su constructor ejecuta super() así que ejecuta Shape(). Desde square, su método foo() ejecuta super.foo() aí que ejecuta foo() de la clase Shape. Desde square, podemos ejecutar square.foo() así directamente. La opción d realiza estos 3 pasos.

Pregunta 30

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál de los siguientes fragmentos de código generará una ClassCastException en tiempo de ejecución?

```
// Opción a
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        A a = (A) obj;
    }
}

// Opción b
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        B obj = new B();
        A a = (A) obj;
    }
}

// Opción c
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        B b = (B) obj;
    }
}

// Opción d
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A ob = new A();
        B b = (B) ob;
    }
}
```

Seleccione una:

- a. a
- b. b
- c. c
- d. d ✓

Justificación: Un downcast a B solo funciona en tiempo de ejecución si el objeto real es un B. En d el objeto es un A puro, así que se lanza ClassCastException. La opción c es la trampa: también hace un downcast, pero el objeto real es un B y funciona. a y b son upcasts, que siempre son seguros.

Fuente: Java.pdf #38

La respuesta correcta es: d

Mi justificación
El incorrecto es d ya que el objeto ob es de tipo A (padre) y el código pretende crear B b con un downcast pero esto no es posible ya que el objeto creado es de tipo A (padre). Entonces el downcast es inválido.

Pregunta 31

Parcialmente correcta

Se puntúa 0,33 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which three are valid? (Choose three).

```
class ClassA{}
class ClassB extends ClassA{}
class ClassC extends ClassA{}
And:
ClassA p0 = new ClassA();
ClassB p1 = new ClassB();
ClassC p2 = new ClassC();
ClassA p3 = new ClassB();
ClassA p4 = new ClassC();
```

Seleccione una o más de una:

- [ ] a. p1 = p2;
- [x] b. p2 = (ClassC)p4; ✓
- [x] c. p2 = p4; ✗
- [x] d. p1 = (ClassB)p3; ✓
- [ ] e. p0 = p1;
- [ ] f. p2 = (ClassC)p1;

Ha seleccionado correctamente 2.

Justificación: p0 = p1 es un upcast implícito. (ClassC)p4 y (ClassB)p3 son downcasts válidos porque los objetos reales son ClassC y ClassB. p2 = p4 necesita un cast explícito, y (ClassC)p1 es un cast entre clases hermanas que no compila.

Fuente: Java D #13 · también aparece en: Java Sesiones #42

Las respuestas correctas son: p2 = (ClassC)p4;, p1 = (ClassB)p3;, p0 = p1;

Mi justificación
XXX a) es inválida porque es hacer B = C, y como son hijos diferentes de A, no es válido. VVV b) es válido porque estamos haciendo C = C, el cast está demás pero es válido. VVV c) es válido porque estamos haciendo C = C, aun si sus tipos de referencia son diferentes. VVV d) es válido porque estamos haciendo B = B, aun si sus tipos de referencia son diferentes. XXX e) es inválido porque es hacer A = B sin casteo, ósea un downcast inválido. XXX a) es inválida porque es hacer B = C, y como son hijos diferentes de A, no es válido.

Pregunta 32

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which is true?

```
5.  class Building { }
6.  public class Barn extends Building {
7.      public static void main(String[] args) {
8.          Building build1 = new Building();
9.          Barn barn1 = new Barn();
10.         Barn barn2 = (Barn) build1;
11.         Object obj1 = (Object) build1;
12.         String str1 = (String) build1;
13.         Building build2 = (Building) barn1;
14.     }
15. }
```

Seleccione una:

- a. If line 10 is removed, the compilation succeeds.
- b. If line 11 is removed, the compilation succeeds.
- c. If line 12 is removed, the compilation succeeds. ✓
- d. If line 13 is removed, the compilation succeeds.
- e. More than one line must be removed for compilation to succeed.

Justificación: El único error de compilación es el cast de Building a String de la línea 12: son tipos sin relación de herencia. El cast de la línea 10 (Building a Barn) compila y solo falla en tiempo de ejecución, así que quitar la línea 10 no basta para compilar.

Fuente: Java Sesiones #21 · también aparece en: Quiz Desarrollo APX 1 #8

La respuesta correcta es: If line 12 is removed, the compilation succeeds.

?

Mi justificación
Es la línea 12 ya que el objeto Building b1 no se puede castear como (String) ya que String no es superclase de build.

Pregunta 33

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código: ¿Cuál es el resultado de la ejecución del programa?

```
interface Omega {}

class Gamma {}

class Epsilon extends Gamma implements Omega {}

public class Test {
    public static void main(String[] args) {
        Gamma g = new Epsilon();
        Omega o = (Omega) g;  // Corrected line
        Epsilon e = (Epsilon) g; // Corrected line
        Omega m= e;
    }
}
```

Seleccione una:

- a. An exception is thrown at runtime.
- b. No option is correct.
- c. Compilation fails.
- d. The code runs with no output. ✓

Justificación: g apunta a un objeto Epsilon, que extiende Gamma e implementa Omega, así que ambos casts son válidos en compilación y en ejecución. Asignar un Epsilon a una referencia Omega no requiere cast. Como no hay ningún println, el programa no imprime nada.

Fuente: Java.pdf #181 · también aparece en: Cuestionario APX V3 #18

Mi justificación
No hay errores en el código, compila sin output ya que las clases no imprimen nada.

Pregunta 34

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Cuál es el resultado?

```
import java.util.ArrayList;
public class Test {
    public static void main(String[] args) {
        String v = new String();
        StringBuffer sb = new StringBuffer();
        class MyList extends ArrayList {
        }
        System.out.println((v instanceof String) &&
            (sb instanceof Object) && (new MyList() instanceof ArrayList));
    }
}
```

Seleccione una:

- a. false
- b. true ✓
- c. The code runs with no output.
- d. Compilation fails.
- e. An exception is thrown at runtime.
- f. null

Justificación: v es un String, sb (un StringBuffer) es un Object y MyList, una clase local que extiende ArrayList, es un ArrayList. Las tres expresiones instanceof dan true. Declarar una clase local dentro de main es legal.

Fuente: Simulación APX V4 #19 · también aparece en: Java.pdf #54

La respuesta correcta es: true

?

Mi justificación
Las 3 condiciones para imprimir true son: 1. El objeto v es instancia de String, esto es true porque desde su inicialización es String v = new String(). 2. El objeto StringBuffer sb es instancia de Object, esto es true porque los stringbuilder son clases heredadas de Object. 3. El objeto MyList es instancia de ArrayList. Como MyList es clase heradada de ArrayList, da true. True&&true&&true = true.

Pregunta 35

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
class Foo{
    public void addFive(){ a += 5; System.out.println("f");}
}
class Bar extends Foo{
    public int a = 8;
    public void addFive(){ this.a += 5; System.out.println("b");}
}
Invoked with:
    Foo f = new Bar();
    f.addFive();
    System.out.println(f.a);
```

Seleccione una:

- a. b 8
- b. b 13 ⊗
- c. f 13
- d. An exception is thrown at runtime.
- e. b 3
- f. f 8
- g. Compilation fails
- h. f 3

Justificación: Foo no declara ningún campo a, así que ni su addFive() ni la expresión f.a (con f de tipo Foo) compilan: los campos se resuelven por el tipo declarado, no por el objeto real. El distractor 'b 13' supone que el campo de Bar es visible desde Foo.

Fuente: Java DE #6

La respuesta correcta es: Compilation fails

Mi justificación
Al crear un new Bar() tenemos que a = 8, al ejecutar addFive() se le suma 5 y da 13, además se imprime “b” Después se imprime f.a el cual es 13 así que se imrpime “b 13”.

Pregunta 36

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which two, independently, will allow Sub to compile? (Choose two)

```
1.  class Super {
2.      private int a;
3.      protected Super(int a) { this.a = a; }
4.  }
...
11. class Sub extends Super {
12.     public Sub(int a) { super(a); }
13.     public Sub() { this.a = 5; }
14. }
```

Seleccione una o más de una:

- a. Change line 2 to: public int a; ✗
- b. Change line 13 to: public Sub() { super(5); } ✓
- c. Change line 2 to: protected int a;
- d. Change line 13 to: public Sub() { this(5); }
- e. Change line 13 to: public Sub() { super(a); }

Justificación: La línea 13 tiene dos errores: accede a «a», que es private en Super, y como no llama a otro constructor el compilador inserta super(), que no existe porque Super solo declara Super(int). b (super(5)) y d (this(5), que delega en Sub(int) y este en super(a)) resuelven ambos a la vez. Cambiar la visibilidad de «a» (a, c) arregla el acceso, pero sigue faltando el constructor sin argumentos de Super. En e, super(a) usa el campo privado «a» (y además un campo de instancia no puede usarse antes de que termine la llamada a super). Verificado con javac 21.

Fuente: Diagnóstico APX 5 #69 · también aparece en: Examen Diagnóstico APX 4 (PDF), pág. 9

Las respuestas correctas son: Change line 13 to: public Sub() { super(5); }, Change line 13 to: public Sub() { this(5); }

Mi justificación
Las líneas que se deben ejecutar son Public Sub(int a) {super(a)}. El problema está en que super(a) ósea Super.a es private así que no podemos acceder a él. Para esto podemos poner int a de Super como public. Public Sub() {this.a = 5}. El atributo a no existe en la clase Sub pero si en Super, ósea que si cambiamos this.a por super(a), sí compila. Las correctas son a y b.

Pregunta 37

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Which five methods, inserted independently at line 5, will compile? (Choose five).

```
1 public class Blip{
2       protected int blipvert(int x){ return 0 ;}
3 }
4 class Vert extends Blip{
5       //insert code here
6 }
```

Seleccione una o más de una:

- a. protected long blipvert(long x) { return 0; } ✓
- b. public int blipvert(int x) { return 0; } ✓
- c. private int blipvert(long x) { return 0; } ✓
- d. protected long blipvert(int x, int y) { return 0; } ✓
- e. protected long blipvert(int x) { return 0; }
- f. protected int blipvert(long x) { return 0; } ✓
- g. private int blipvert(int x) { return 0; }

Justificación: Las opciones con parámetros distintos (long, o int y int) son sobrecargas y admiten cualquier modificador y tipo de retorno (a, c, d, f). public int blipvert(int) es una sobrescritura válida que amplía el acceso (b). Fallan e (sobrescritura con retorno long, incompatible con int) y g (private reduce el acceso protected); javac: 'cannot override'.

Fuente: Java D #6

Las respuestas correctas son: protected long blipvert(long x) { return 0; }, public int blipvert(int x) { return 0; }, private int blipvert(long x) { return 0; }, protected long blipvert(int x, int y) { return 0; }, protected int blipvert(long x) { return 0; }

Mi justificación
Para que sea válido debemos cumplir las siguientes reglas: 1. Que el método de acceso sea protected o de mayor acceso. 2. Que el valor de retorno sea int como el del padre. 3. Si no tiene el mismo valor de retorno, que tenga más parámetros que el método original. Las que no cumplen son: XXX e) porque el valor de retorno es long, y debe ser int. XXX g) porque el acceso se restringió de public a private. VVV Las demás cumplen con las 3 reglas.

Pregunta 38

Parcialmente correcta

Se puntúa 0,33 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Which three lines will compile and output "Right on!"?

```
13.  public class Speak {
14.      public static void main(String[] args) {
15.          Speak speakIT = new Tell();
16.          Tell tellIt = new Tell();
17.          speakIT.tellItLikeItIs();
18.          (Truth) speakIT.tellItLikeItIs();
19.          ((Truth) speakIT).tellItLikeItIs();
20.          tellIt.tellItLikeItIs();
21.          (Truth) tellIt.tellItLikeItIs();
22.          ((Truth) tellIt).tellItLikeItIs();
23.      }
24.  }

class Tell extends Speak implements Truth {
    @Override
    public void tellItLikeItIs() {
        System.out.println("Right on!");
    }
}

interface Truth {
    public void tellItLikeItIs();
}
```

Seleccione una o más de una:

- [ ] a. Line 17
- [x] b. Line 18 ✗
- [x] c. Line 19 ✓
- [x] d. Line 20 ✓
- [ ] e. Line 21
- [ ] f. Line 22

Ha seleccionado correctamente 2.

Justificación: La línea 17 falla porque la referencia es de tipo Speak, que no declara el método. En las líneas 18 y 21 el cast se aplica al resultado void de la llamada, porque la llamada tiene mayor precedencia, y eso no es una sentencia válida. En las líneas 19 y 22 el cast va entre paréntesis antes de invocar el método, y en la línea 20 la referencia ya es de tipo Tell.

Fuente: Java Sesiones #44 · también aparece en: Java DE #24, Java D #23

Las respuestas correctas son: Line 19, Line 20, Line 22

Mi justificación
La respondí al azar, estaba muy complicada de leer.

Pregunta 39

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Dado el siguiente código. ¿Cuál es el resultado?

```
class A{
    int x = 8;
    public String ejemplo() {
        x+=2;
        return "Valor de la clase A";
    }
}
class B extends A{
    int x = 12;
    public String ejemplo() {
        x+=4;
        return "Valor de la clase B";
    }
}
class C extends B{
    int x = 18;
    public String ejemplo() {
        x+=6;
        return "Valor de la clase C";
    }
}
public class HerenciaVar extends C{
    public static void main(String[] args) {
        A ac = new C();
        System.out.println(ac.ejemplo());
        System.out.println(ac.x + 20);
    }
}
```

Seleccione una:

a. Valor de la clase A 30
b. Valor de la clase B 36
c. Valor de la clase C 28
d. Valor de la clase C 44 ⊗

Justificación: El método ejemplo() está sobrescrito, así que se ejecuta el de C, que incrementa el x de C. Pero los atributos no son polimórficos: ac es de tipo A, por lo que ac.x es el x de A, que sigue valiendo 8, y 8 + 20 = 28. El distractor d (44) supone que ac.x accede al x de C ya incrementado.

Fuente: Quiz Desarrollo APX 2 #19

La respuesta correcta es: Valor de la clase C 28

Mi justificación
Es muy directa. Se crear ac = new C con ac.x = 18. Se ejecuta ac.ejemplo que suma 6 al valor de x, ósea ac.x =24. Se ejecuta ac.x +20 lo cual daría 44.

Pregunta 40

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

What is the result?

```
abstract class Animal {
    private void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Seleccione una:

- a. Animals make noise
- b. Compilation fails. ⊗
- c. The Dog barks
- d. A runtime exception is thrown.

Justificación: Un método private no se hereda ni se sobrescribe, así que Dog.speak() es un método nuevo. Dentro de Animal, animal.speak() se resuelve en compilación al método private de Animal, y por eso imprime "Animals make noise". Si speak() no fuera private, se imprimiría "The Dog barks".

Fuente: Java.pdf #155 · también aparece en: Cuestionario APX V3 #4

La respuesta correcta es: Animals make noise

Mi justificación
No compila porque el método main ocupa un arreglo para los parámetros, no es válido con (strings).

RESULTADO = 30/40 = 7.50